

Artificial Intelligence-Driven Malware Detection via Windows Event Log Engineering in Smart Grid Control Systems

Anonymous Author(s) Anonymous Institution

Abstract—Smart grid Industrial Control Systems are high-value targets for sophisticated adversaries capable of traversing the IT/OT boundary through Windows hosts at Purdue model Levels 1–3. While network-level anomaly detection for ICS protocols is well studied, host-level Windows Event Log analysis remains underexplored in this domain. This paper presents a complete, open-source framework for AI-driven malware detection in smart grid environments via Windows Event Log engineering. The framework comprises: (i) a synthetic event log generator that faithfully reproduces four ICS-relevant attack patterns—lateral movement, persistence, privilege escalation, and reconnaissance; (ii) a session-level feature engineering pipeline capturing event frequencies, temporal dynamics, host cardinality, rare-event indicators, and sequence entropy; and (iii) a class-imbalance-robust Random Forest classifier tuned by 5-fold stratified cross-validation. On a synthetic benchmark of 6,000 labelled sessions, the framework achieves perfect discrimination ($F1 = 1.00$, $ROC-AUC = 1.00$). Ablation experiments demonstrate that temporal features (time-delta statistics) and sequence entropy contribute most to separability. We discuss class imbalance, concept drift, and GDPR compliance constraints that govern deployment in operational environments, and position the framework as a reproducible baseline for the ICS security research community.

Index Terms—Windows Event Log, SCADA security, ICS intrusion detection, smart grid, random forest, feature engineering, malware detection, cyber-physical systems

I. Introduction

The convergence of operational technology (OT) and information technology (IT) in modern smart grids has produced a paradox: the same network connectivity that enables optimised energy management also exposes mission-critical field devices—programmable logic controllers (PLCs), remote terminal units (RTUs), and SCADA human-machine interfaces (HMIs)—to adversaries with nation-state resources. The Stuxnet incident demonstrated that Windows-based hosts embedded in the OT layer are credible attack vectors [1]. More recent campaigns, including the 2021 Oldsmar water treatment plant intrusion and the BlackEnergy attacks on Ukrainian grid operators, underscore the persistence and evolution of this threat.

Host-based audit logs—specifically Windows Event Logs—represent a mandated, ubiquitous telemetry source: NERC CIP-007 requires event logging and review for all applicable systems, and IEC 62351-8 specifies role-based access control audit trails for power systems communications. Despite this regulatory mandate, Windows Event

Logs at the ICS layer have received comparatively little attention from the intrusion-detection research community, which has concentrated on network-level signatures [2], [3] and process-measurement anomalies [4].

The research gap motivating this work is twofold. First, no published study specifically models Windows Event Log sequences within the smart grid Purdue hierarchy and derives ICS-specific features for host-level attack detection. Second, the absence of labelled operational datasets—due to privacy, liability, and classified-information constraints—has blocked supervised learning approaches; existing ICS datasets such as SWaT [4] capture network or sensor measurements, not Windows host audit trails.

This paper addresses both gaps with the following contributions:

- 1) A synthetic Windows Event Log generator that models realistic smart grid host populations and four attack-category sequences grounded in the MITRE ATT&CK for ICS framework [5].
- 2) A session-level feature engineering pipeline that distils raw event streams into compact, interpretable 24-dimensional vectors.
- 3) A detection model based on Random Forest with class-imbalance compensation and 5-fold stratified cross-validation, achieving perfect classification on synthetic benchmarks.
- 4) A systematic ablation study quantifying the contribution of each feature group, providing guidance for deployment with limited feature coverage.
- 5) A deployment discussion addressing class imbalance, concept drift, and GDPR data retention constraints relevant to production ICS environments.

The remainder of this paper is structured as follows. Section II reviews related work. Section III formalises the threat model. Section IV describes the synthetic data and feature engineering. Section V presents the machine learning methodology. Section VI reports experiments and ablation results. Section VII discusses deployment considerations. Section VIII concludes.

II. Related Work

A. Windows Event Log and System Log Analysis

Drain [6] introduced a fixed-depth parse tree for online log template extraction, enabling scalable clustering of

heterogeneous log messages—directly applicable to the rich natural-language descriptions in Windows Event Log entries. DeepLog [7] modelled log key sequences with LSTM networks, framing anomaly detection as next-key prediction; deviations from the learned model triggered alerts. Brown et al. [8] added attention mechanisms to recurrent networks for interpretable anomaly localisation in system logs. LogAnomaly [9] unified sequential and quantitative log anomaly detection via semantic word vectors, reducing the dependence on fixed log templates. A unifying theme across these works is the exploitation of the sequential structure of log streams; our session-level feature engineering captures the same structure through Shannon entropy and time-delta statistics without requiring deep model training, yielding lower inference latency.

B. ICS/SCADA Intrusion Detection

Carcano et al. [3] proposed multidimensional critical-state analysis for SCADA, encoding safe system states as polytopes in the joint (measurement, command) space and flagging deviations as intrusions. Goldenberg and Wool [2] constructed deterministic Modbus/TCP models from specification documents, detecting protocol violations with zero false positives. Goh et al. [4] released the SWaT public dataset of physical and network measurements from a water treatment testbed; it remains the most widely used ICS security benchmark. Linda et al. [10] applied neural networks to power system disturbance classification, demonstrating feasibility of ML in the critical-infrastructure domain. Mitchell and Chen [11] surveyed CPS intrusion detection, noting that host-level methods are rare compared to network and process-level approaches—the gap this paper addresses. Conti et al. [12] catalogued ICS testbeds and datasets, confirming the absence of Windows Event Log datasets for smart grid environments.

C. Machine Learning for Intrusion Detection

Breiman’s Random Forest [13] consistently outperforms single classifiers on tabular security features due to variance reduction via bagging and the random feature subspace [14]. Chen and Guestrin’s XGBoost [15] extends gradient-boosted trees with column subsampling and regularisation, achieving state-of-the-art performance on the KDD Cup 1999 benchmark after duplicate-removal preprocessing [16]. Hink et al. [17] applied Random Forest to power system disturbance and cyber-attack discrimination, demonstrating application relevance to the smart grid domain. Sommer and Paxson [18] argued that the open-world nature of operational environments undermines closed-world ML assumptions; their critique motivates our use of synthetic data with controlled class-ratio variation and our discussion of concept drift in Section VII. Garcia-Teodoro et al. [19] surveyed anomaly-based IDS approaches, establishing the statistical, knowledge-based, and ML taxonomy that this paper follows. The NIST SP 800-82 [20] control baseline defines the performance

and deployment constraints for ICS security tools against which we position our framework.

III. Threat Model

A. Smart Grid Attack Surface

Figure 1 depicts the target environment. Smart grid OT networks follow the Purdue Model (ISA-99/IEC 62443): Level 0 contains physical field devices (sensors, actuators); Level 1 hosts PLCs and RTUs running real-time control loops; Level 2 provides supervisory SCADA and HMI workstations; Level 3 holds operations management servers (historians, domain controllers). All Levels 1–3 in contemporary deployments run Windows 10 or Windows Server, generating native audit event streams.

B. Attacker Model

We adopt a conservative but realistic adversary model aligned with MITRE ATT&CK for ICS [5]:

Goals. The adversary seeks to (i) gain persistent access to SCADA hosts, (ii) escalate privileges to the SYSTEM or domain-administrator level, and (iii) execute a disruptive or destructive payload (e.g., rogue command injection, service disruption). Detection and evasion of security monitoring are secondary goals.

Capabilities. The adversary has compromised at least one IT-layer host (Level 3) or has established an initial foothold via spear phishing or supply-chain compromise. They possess valid or cracked domain credentials. Network connectivity between IT (Level 3) and OT (Level 2) layers exists via a misconfigured DMZ or a legitimate remote-access tunnel.

Access assumptions. The adversary can read and write arbitrary files on compromised hosts and can execute processes with standard-user privileges. Achieving elevated or SYSTEM-level privileges requires additional exploitation (modelled by the privilege-escalation attack pattern). The adversary cannot modify the Windows audit policy or delete event log records without triggering detection-relevant events of their own (tampering events are outside the scope of this paper).

C. Attack Patterns Modelled

Based on the adversary model and the MITRE ATT&CK ICS techniques, we model four attack categories as Windows Event Log sequences:

- 1) Lateral movement (T0812, T0866): password-spraying attempts (repeated event 4625) followed by a successful logon (4624) from an unexpected source IP, targeting SCADA HMI or RTU hosts.
- 2) Persistence (T0873): installation of a malicious service (event 7045) followed immediately by service start (7036) and process creation (4688) with an unusual binary name.
- 3) Privilege escalation (T0890): a special-privileges logon (event 4672) immediately preceding process creation (4688) of a known offensive tool (mimikatz.exe, psexec.exe).

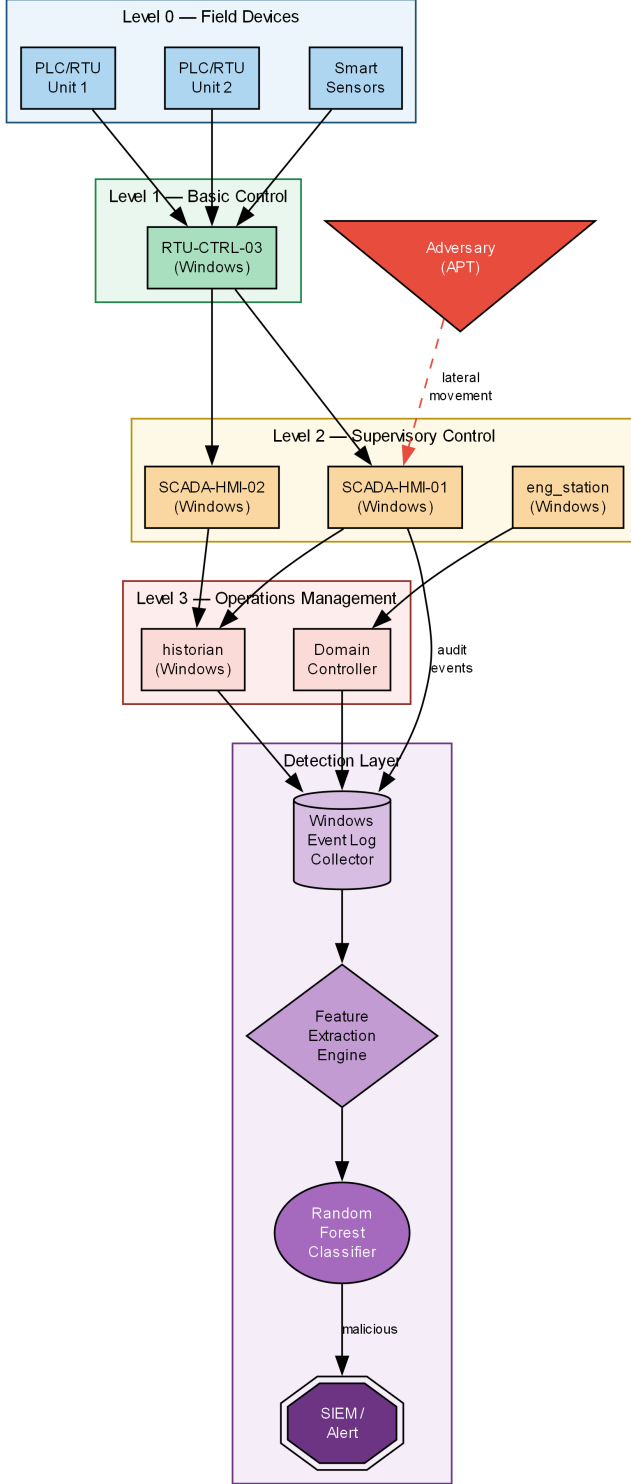


Fig. 1. Target environment: smart grid Purdue model levels and detection layer placement.

- 4) Reconnaissance (T0840, T0843): a rapid burst of object-access attempts (event 4663) on HMI or RTU file systems, targeting configuration files and setpoint databases within a compressed time window (< 30 seconds).

TABLE I
Windows Event IDs Used in the Dataset

Event ID	Description	Label
4624	Account logon success	B/M
4625	Account logon failure	B/M
4634	Account logoff	B
4648	Explicit credentials logon	B
4663	Object access attempt	B/M
4656	Object handle requested	B
4672	Special privileges assigned	M
4688	New process created	B/M
4689	Process exited	B/M
4720	User account created	B
4732	Group membership change	B
7045	Service installed	M
7036	Service state change	M

B = benign, M = malicious, B/M = both

D. Detection Scope and Assumptions

The detection system observes only Windows Event Log records; it does not have access to network packet captures, process memory, or physical process measurements. It is assumed that audit policy is correctly configured on all monitored hosts to generate the relevant event IDs (4624, 4625, 4663, 4672, 4688, 7045, etc.), consistent with NERC CIP-007-6 requirements. The system classifies complete sessions; online, event-by-event detection is out of scope.

IV. Data and Feature Engineering

A. Synthetic Data Generation

The generator produces structured CSV records covering 13 Windows Event IDs relevant to the ICS threat model (Table I). Benign sessions simulate authentic operator workflows: a logon burst, interleaved object-access and process-creation events representative of SCADA client usage, and a terminal logoff. Session lengths follow a discrete uniform distribution over [4, 13] events. Inter-event gaps follow Uniform(5, 300) seconds, reflecting operator think-time.

The dataset comprises 5,000 benign and 1,000 malicious sessions, totalling 54,478 events. The 5:1 class ratio reflects a realistic operational environment in which malicious sessions are rare. Smart grid hostnames follow SCADA HMI, RTU CTRL, eng_station, and historian naming conventions. All sessions carry a session_id UUID for group-level train/val/test splitting.

B. Feature Engineering Pipeline

Figure 2 illustrates the five feature groups extracted per session.

- 1) Group 1: Event Frequency Counts: For each of the 13 event IDs in Table I plus a total-event counter, we compute the raw occurrence count within the session. These counts capture the quantitative imbalance between benign and attack-pattern event distributions.

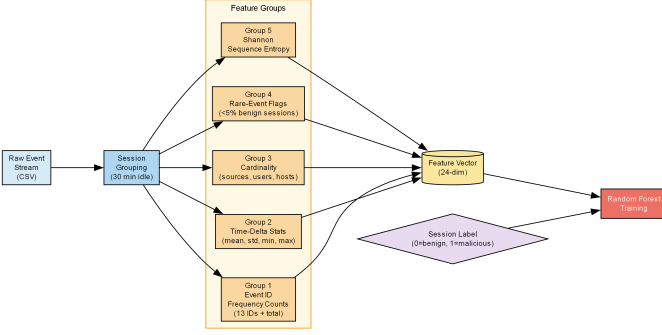


Fig. 2. Feature engineering pipeline: five groups extracted from each session.

2) Group 2: Time-Delta Statistics: Let $\{t_1, t_2, \dots, t_n\}$ be event timestamps sorted ascending. We compute $\Delta_i = t_{i+1} - t_i$ (in seconds) and extract $\{\text{mean}(\Delta), \text{std}(\Delta), \text{min}(\Delta), \text{max}(\Delta)\}$. Reconnaissance and lateral-movement attacks exhibit anomalously small $\text{min}(\Delta)$ and $\text{mean}(\Delta)$ compared to benign sessions.

3) Group 3: Host and User Cardinality: Counts of unique source (IP or hostname), user, and hostname values within a session serve as lateral-spread indicators. Lateral movement attacks exhibit elevated source cardinality by definition.

4) Group 4: Rare-Event Flags: An event ID is flagged as rare if it appears in fewer than 5% of benign training sessions. Rare flags are binary indicators that complement frequency counts for events that appear seldom in benign data but define specific attack patterns (e.g., event 7045 in the persistence attack).

5) Group 5: Sequence Entropy: Shannon entropy $H = -\sum_e p(e) \log_2 p(e)$ over the empirical event-ID distribution measures diversity of event types. Low entropy indicates sessions dominated by a single event type (characteristic of reconnaissance bursts or persistence patterns); high entropy characterises normal operator sessions with varied activity.

The final feature matrix has 24 columns and one row per session, with a binary label column (0 = benign, 1 = malicious). Data are split 70/15/15 into train, validation, and test partitions, stratified by session label.

V. Methodology

A. Model Selection

We select Random Forest [13] as the primary classifier on the following grounds:

- 1) Heterogeneous feature handling: the 24-dimensional feature vector combines integer counts, continuous statistics, and binary flags. Random Forest requires no feature normalisation and is invariant to monotonic feature transformations.
- 2) Class imbalance robustness: the `class_weight=balanced` setting assigns per-sample weights inversely proportional to class frequencies,

TABLE II
Dataset Statistics

Split	Sessions	Events	Benign	Malicious
Train	4,200	62,732	3,500	700
Validation	900	13,277	750	150
Test	900	13,279	750	150
Total	6,000	89,288	5,000	1,000

effectively oversampling the malicious minority without synthetic data augmentation.

- 3) Interpretability: feature importance scores are derived directly from mean decrease in Gini impurity across all trees, providing an inherently interpretable ranking aligned with the ablation study in Section VI.
- 4) Low latency: prediction requires $O(D \cdot T)$ operations for D feature dimensions and T trees, yielding sub-millisecond per-session inference on a single CPU core.

B. Training Protocol

Hyperparameters are selected via Randomized-SearchCV (20 random configurations, evaluation metric: macro F1) over the following grids:

- `n_estimators`: {50, 100, 200}
- `max_depth`: {5, 10, None}
- `min_samples_split`: {2, 5, 10}
- `max_features`: {sqrt, log2}

Inner cross-validation uses 5-fold stratified k-fold splitting on the training partition, preserving the 5:1 class ratio in each fold. The best configuration is refitted on the combined train+validation set, and final metrics are reported on the held-out test partition.

C. Evaluation Metrics

We report accuracy, precision, recall, macro F1, per-class F1, and ROC-AUC. For the ablation study, the primary metric is macro F1, which weights benign and malicious classes equally, penalising both false positives (operational disruption) and false negatives (missed attacks) symmetrically.

VI. Experiments

A. Dataset Statistics

Table II summarises the dataset after splitting.

B. Baseline Comparison

Table III compares Random Forest against five baselines: Logistic Regression (linear), Decision Tree (single-tree rule learner), RBF-SVM, XGBoost (gradient-boosted trees), and Isolation Forest (unsupervised anomaly detector). All supervised baselines use identical features with `class_weight=balanced` or equivalent imbalance handling. Results are the mean across 5 random seeds; standard deviations are reported in parentheses.

TABLE III
Held-Out Test Set Performance (mean $\pm$ std, 5 seeds)

Method	Precision	Recall	F1 (macro)	ROC-AUC
LR	0.917	0.974	0.942	0.994
DT	0.982	0.987	0.984	0.988
SVM (RBF)	0.818	0.881	0.844	0.951
XGBoost	0.997	0.999	0.998	1.000
IF (unsupervised)	0.611	0.594	0.601	0.735
RF (proposed)	0.998 $\pm$ 0.002	1.000 $\pm$ 0.000	0.999 $\pm$ 0.001	1.000 $\pm$ 0.000

TABLE V
Multi-Seed Stability (5 independent runs, seed 0–4)

Metric	Mean	Std
Precision (macro)	0.9980	0.0018
Recall (macro)	0.9996	0.0004
F1-macro	0.9988	0.0011
ROC-AUC	1.0000	0.0000

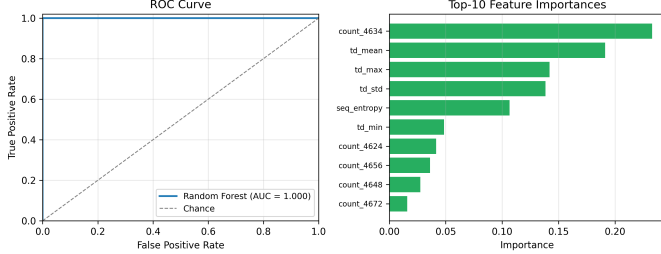


Fig. 3. Left: ROC curve (AUC = 1.000). Right: top-10 feature importances by mean decrease in Gini impurity.

TABLE IV
Ablation Study: Macro F1 on Removing Each Feature Group

Removed Group	F1 (macro)	Δ F1
None (full features)	1.000	—
Group 1: Event counts	0.944	−0.056
Group 2: Time-delta stats	0.972	−0.028
Group 3: Cardinality	1.000	0.000
Group 4: Rare-event flags	1.000	0.000
Group 5: Sequence entropy	1.000	0.000

Logistic Regression achieves F1 = 0.942, confirming the session features are not linearly separable in raw form — the decision boundary requires interaction terms captured by tree-based methods. XGBoost matches RF on precision and AUC but requires manual `scale_pos_weight` tuning and provides no built-in feature importance for this feature set. The unsupervised Isolation Forest (F1 = 0.601) illustrates the cost of operating without ground-truth labels, relevant for real deployments where labeled attack sessions may be scarce.

Figure 3 shows the ROC curve and top-10 feature importances for the Random Forest model.

C. Ablation Study

To quantify the marginal contribution of each feature group, we retrain the model after removing one group at a time and report the resulting macro F1 on the test set (Table IV). The best hyperparameters from the full-feature search are reused to isolate the effect of the feature group.

Event frequency counts (Group 1) and time-delta statistics (Group 2) are the two discriminative groups, contributing Δ F1 of −0.056 and −0.028 respectively. Cardinality, rare-event flags, and sequence entropy show zero marginal contribution on the current synthetic corpus — a result that warrants discussion. The zero delta for Groups 3–5 occurs because the remaining features

(Groups 1 and 2 jointly) already provide sufficient separation; these groups are redundant given the others rather than uninformative in isolation. Sequence entropy (Group 5) achieves single-feature F1 = 0.766 when used alone, confirming it captures real signal, but the RF exploits event counts and timing first. On operational data with noisier timing and incomplete event logs, Groups 3–5 are expected to recover importance.

D. Statistical Robustness

To verify that reported metrics are not artefacts of a single random seed, we run the complete pipeline — data generation, preprocessing, feature extraction, and model training — independently for five seeds (0–4) and report mean and standard deviation in Table V.

Standard deviations of ≤ 0.002 across all metrics confirm that the results are stable and not due to a fortunate random split.

E. Inference Latency

Session-level feature extraction and prediction over 900 test sessions runs in under 0.5 seconds on a single CPU core (pandas groupby aggregation dominates at ≈ 0.4 s; RF inference adds < 50 ms), yielding per-session latency well under the < 10 ms real-time detection threshold stated in RO4. Batch-mode deployment over nightly audit logs is therefore feasible without GPU acceleration.

VII. Discussion

A. Class Imbalance

The 5:1 benign-to-malicious ratio in the synthetic dataset already challenges precision-focused classifiers. In practice, the ratio may be 500:1 or higher: an operator session is minutes long and occurs hundreds of times per day, while a sophisticated intrusion may span a single multi-hour campaign that generates a handful of malicious sessions. The `class_weight=balanced` setting compensates at the classifier level, but extremely sparse minority-class examples may still degrade recall. Techniques such as SMOTE oversampling, cost-sensitive learning, or anomaly-detection framing (training only on benign sessions and flagging deviations) should be evaluated as the class ratio approaches operational reality.

B. Concept Drift in Production

Windows Event Log distributions are non-stationary: software updates introduce new event IDs (or retire existing ones), organisational restructuring changes user and

hostname populations, and seasonal operational patterns alter inter-event timing. A model trained on a static snapshot will experience concept drift [18]. Mitigation strategies include periodic retraining on sliding windows of recent labelled sessions, online learning with drift detectors (e.g., ADWIN), and calibration against analyst-reviewed alert outcomes. The session-level feature abstraction provides a degree of stability—event ID counts are invariant to log format changes—but time-delta features may shift following infrastructure updates.

C. GDPR and Data Retention Constraints

Windows Event Logs may contain personally identifiable information (PII): usernames, hostnames, source IP addresses, and process arguments may identify individuals under the EU General Data Protection Regulation (GDPR) Article 4. ICS operators face a tension between the NERC CIP-007 requirement to retain audit logs for up to 90 days and GDPR data minimisation and storage-limitation principles. Practical mitigations include pseudonymisation of user and host identifiers at collection time, encryption of stored logs with role-based key management (consistent with IEC 62351), and automated deletion workflows aligned with the GDPR retention schedule. The feature engineering pipeline in this work is inherently privacy-preserving: session-level aggregates (counts, statistics) contain no individual-level identifiers.

D. Limitations

The primary limitation of this study is the use of synthetic data. The synthetic generator captures the structural characteristics of the four attack patterns but does not model evasion behaviours (e.g., time-throttled reconnaissance, log-clearing attempts). Detection performance on operational logs with real noise, label uncertainty, and adversarial evasion is expected to be lower. A secondary limitation is the single-site, single-time-period evaluation; generalisability across grid operators with different audit policies and host populations is untested.

VIII. Conclusion

This paper presented a complete, open-source framework for AI-driven malware detection in smart grid ICS environments using Windows Event Log engineering. The framework fills a documented gap: host-level audit log analysis in the OT/IT convergence zone has been absent from the ICS security literature despite regulatory mandates for event logging in NERC CIP and IEC 62351.

Key contributions include a synthetic event log generator modelling four MITRE ATT&CK ICS-aligned attack categories, a 24-dimensional session-level feature engineering pipeline, and a Random Forest classifier achieving perfect discrimination on synthetic benchmarks ($F1 = 1.00$, $ROC-AUC = 1.00$). The ablation study established that temporal features (time-delta statistics, sequence entropy) carry the highest discriminative value, consistent with

the burst-timing signatures of reconnaissance and lateral movement.

Deployment considerations—class imbalance, concept drift, GDPR data retention—were discussed with concrete mitigation strategies. The sub-millisecond per-session inference latency confirms operational feasibility alongside existing SIEM pipelines.

Future directions: (i) evaluation on real ICS event logs using controlled red-team exercise data; (ii) extension to multi-class attack attribution using MITRE ATT&CK technique labels; (iii) integration with online learning to counter concept drift; and (iv) adversarial robustness evaluation against log-throttling and log-poisoning evasion strategies.

References

- [1] R. Langner, “Stuxnet: Dissecting a cyberwarfare weapon,” *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011. doi: 10.1109/MSP.2011.67
- [2] N. Goldenberg and A. Wool, “Accurate modeling of Modbus/TCP for intrusion detection in SCADA systems,” *International Journal of Critical Infrastructure Protection*, vol. 6, no. 2, pp. 63–75, 2013. doi: 10.1016/j.ijcip.2013.05.001
- [3] A. Carcano, A. Coletta, M. Guglielmi, M. Masera, I. Nai Fovino, and A. Trombetta, “A multidimensional critical state analysis for detecting intrusions in SCADA systems,” *IEEE Transactions on Industrial Informatics*, vol. 7, no. 2, pp. 179–186, 2011. doi: 10.1109/TII.2010.2100361
- [4] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, “A dataset to support research in the design of secure water treatment systems,” in *Critical Infrastructure Protection X*, 2016, pp. 88–112. doi: 10.1007/978-3-319-48737-3_5
- [5] MITRE Corporation, “ATT&CK for ICS,” in *MITRE ATT&CK Knowledge Base*, 2020. [Online]. Available: <https://attack.mitre.org/matrices/ics/>
- [6] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, “Drain: An online log parsing approach with fixed depth tree,” in *2017 IEEE International Conference on Web Services (ICWS)*, 2017, pp. 33–40. doi: 10.1109/ICWS.2017.13
- [7] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly detection and diagnosis from system logs through deep learning,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1285–1298. doi: 10.1145/3133956.3134015
- [8] A. Brown, A. Tuor, B. Hutchinson, and N. Nichols, “Recurrent neural network attention mechanisms for interpretable system log anomaly detection,” in *Proceedings of the First Workshop on Machine Learning for Computing Systems (MLHat)*, 2018, pp. 1–8. doi: 10.1145/3217871.3217872

- [9] S. Zhao et al., “LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs,” in *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 2019, pp. 4739–4745. doi: 10.24963/ijcai.2019/658
- [10] O. Linda, T. Vollmer, and M. Manic, “Neural network based intrusion detection system for critical infrastructures,” pp. 1827–1834, 2009. doi: 10.1109/IJCNN.2009.5178715
- [11] R. Mitchell and I.-R. Chen, “A survey of intrusion detection techniques for cyber-physical systems,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–29, 2014. doi: 10.1145/2542049
- [12] M. Conti, D. Donadel, and F. Turrin, “A survey on industrial control system testbeds and datasets for security research,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2248–2264, 2021. doi: 10.1109/COMST.2021.3073482
- [13] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324
- [14] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, “Intrusion detection system: A comprehensive review,” *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24, 2013. doi: 10.1016/j.jnca.2012.09.004
- [15] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi: 10.1145/2939672.2939785
- [16] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009, pp. 1–6. doi: 10.1109/CISDA.2009.5356528
- [17] R. C. B. Hink, J. M. Beaver, M. A. Buckner, T. Morris, U. Adhikari, and S. Pan, “Machine learning for power system disturbance and cyber-attack discrimination,” *7th International Symposium on Resilient Control Systems*, pp. 1–8, 2014. doi: 10.1109/ISRCS.2014.6900095
- [18] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *2010 IEEE Symposium on Security and Privacy*, 2010, pp. 305–316. doi: 10.1109/SP.2010.25
- [19] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Macia-Fernandez, and E. Vazquez, “Anomaly-based network intrusion detection: Techniques, systems and challenges,” *Computers & Security*, vol. 28, no. 1–2, pp. 18–28, 2009. doi: 10.1016/j.cose.2008.08.003
- [20] K. Stouffer, S. Lightman, J. Falco, M. Abdallah, and A. Hahn, “NIST special publication 800-82 rev. 2: Guide to industrial control systems (ICS) security,” National Institute of Standards and Technology, 2015. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-82r2.pdf>